Mini-samouczek Ady – pomoc w realizacji projektu

1. Najprostszy program w Adzie – procedura główna

```
with Ada.Text_IO;
use Ada.Text_IO;

procedure Main is
begin
   Put_Line("Hello Ada!");
end Main;
```

2. Podstawowe konstrukcje

Zmienne i podstawowe typy:

```
Liczba : Integer := 10;
Nazwa  : String := "Robot";
Gotowy : Boolean := True;
```

Instrukcja warunkowa:

```
if Liczba > 5 then
   Put_Line("Dużo");
elsif Liczba = 5 then
   Put_Line("Dokładnie 5");
else
   Put_Line("Mało");
end if;
```

Pętla:

```
for I in 1 .. 5 loop
   Put_Line("Iteracja");
end loop;
```

Pętla nieskończona:

```
loop
   -- instrukcje

   exit when Warunek_Konca;
end loop;
```

3. Wypisywanie liczb

Do zamiany liczby na tekst można użyć 'Image.

```
Put_Line("Robot nr" & Integer'Image(Id));
```

Dla:

```
Id := 3;
```

wynik będzie podobny do:

```
Robot nr 3
```

Magdalena Godlewska

4. Rekordy

Do przechowywania informacji o dostawie lub zamówieniu warto wykorzystać record.

```
type Dostawa is record
   Id    : Integer;
   Towar : Character;
   Ilosc : Integer;
end record;
```

Utworzenie wartości:

```
D : Dostawa;

D.Id    := 1;
D.Towar := 'A';
D.Ilosc := 10;
```

Można również zapisać:

```
D := (
   Id    => 1,
   Towar => 'A',
   Ilosc => 10
);
```

5. Zadania – task

Zadanie w Adzie działa współbieżnie z innymi zadaniami.

Najprostszy przykład - deklaracja:

```
task Robot;
```

oraz jego implementacja:

```
task body Robot is
begin
   Put_Line("Robot rozpoczyna pracę");

   for I in 1 .. 5 loop
      Put_Line("Robot pracuje");
      delay 1.0;
   end loop;

   Put_Line("Robot kończy pracę");
end Robot;
```

Procedura główna oraz zadanie Robot wykonują się współbieżnie.

6. Kilka zadań tego samego typu

W projekcie zazwyczaj jest potrzeba kilku procesów realizujących dane zadanie np. kilku robotów.

Najwygodniej wtedy zdefiniować typ zadania:

```
task type Robot_Type;
```

Następnie:

```
task body Robot_Type is
begin
   loop
      Put_Line("Robot działa");
      delay 1.0;
   end loop;
end Robot_Type;
```

Można wtedy utworzyć kilka robotów:

```
Robot1 : Robot_Type;
Robot2 : Robot_Type;
Robot3 : Robot_Type;
```

Każdy działa niezależnie jako osobny proces.

7. Zadanie z parametrem identyfikującym

Zadanie może posiadać parametry, np. id robota, żeby wiedzieć, który robot realizuje zadanie:

```
task type Robot_Type(Id : Integer);
task body Robot_Type is
begin
   Put_Line("Start robota" & Integer'Image(Id));
end Robot_Type;
```

Utworzenie robotów:

```
Robot1 : Robot_Type(1);
Robot2 : Robot_Type(2);
Robot3 : Robot_Type(3);
```

8. delay

Instrukcja:

```
delay 2.0;
```

wstrzymuje dane zadanie na około 2 sekundy.

Przykład:

```
Put_Line("Rozpoczynam transport");
delay 3.0;
Put_Line("Transport zakończony");
```

delay nie zatrzymuje całego programu. W tym czasie inne procesy mogą nadal realizować swoje zadania.

9. Spotkania – entry i accept

Spotkanie pozwala dwóm zadaniom zsynchronizować się (spotkać).

Jedno zadanie deklaruje entry (wejście):

```
task Magazyn is
   entry Przyjmij(Ilosc : Integer);
end Magazyn;
```

Drugie wywołuje to entry, aby „wejść” do zadania innego procesu:

```
Magazyn.Przyjmij(10);
```

Magazyn obsługuje spotkanie poprzez accept:

```
task body Magazyn is
begin
   loop
      accept Przyjmij(Ilosc : Integer) do
         Put_Line(
            "Przyjęto" & Integer'Image(Ilosc) & " szt."
         );
      end Przyjmij;
   end loop;
end Magazyn;
```

Spotkanie następuje wtedy, gdy:

- jedno zadanie wywołuje entry,
- drugie dochodzi do odpowiadającego mu accept.

10. Przekazywanie danych podczas spotkania

Parametry mogą działać podobnie, jak w procedurach i być przekazywane miedzy procesami.

```
entry Przyjmij(Towar : Character; Ilosc : Integer);
```

Wywołanie:

```
Magazyn.Przyjmij('A', 10);
```

Obsługa:

```
accept Przyjmij(Towar : Character; Ilosc : Integer) do

   Put_Line(
      "Towar " & Towar &
      ", liczba:" & Integer'Image(Ilosc)
   );

end Przyjmij;
```

11. Parametry out

Spotkanie może również zwrócić wynik (jak procedura)

Deklaracja:

```
entry Sprawdz_Stan(
   Towar : Character;
```

```
   Ilosc : out Integer
);
```

Wywołanie:

```
Stan : Integer;

Magazyn.Sprawdz_Stan('A', Stan);
```

Po spotkaniu zmienna Stan zawiera wartość ustawioną przez magazyn.

Przykład:

```
accept Sprawdz_Stan(
   Towar : Character;
   Ilosc : out Integer
) do
   Ilosc := Stan_A;
end Sprawdz_Stan;
```

12. Spotkanie selektywne – select ... or

Zadanie może obsługiwać różne rodzaje komunikatów:

```
select

   accept Przyjmij(Ilosc : Integer) do
      Put_Line("Przyjmuję towar");
   end Przyjmij;

or

   accept Sprawdz_Stan(Wynik : out Integer) do
      Wynik := Stan;
   end Sprawdz_Stan;

end select;
```

Ada wybierze jedno ze spotkań, które może zostać w danym momencie wykonane.

13. select ... or delay

Zadanie może oczekiwać na spotkanie tylko przez określony czas.

```
select

   accept Transportuj(Ilosc : Integer) do
      Put_Line("Rozpoczynam transport");
   end Transportuj;

or

   delay 3.0;
   Put_Line("Robot od 3 sekund nie ma pracy");

end select;
```

Jeżeli w ciągu 3 sekund pojawi się zgłoszenie Transportuj, zostanie ono obsłużone.

Jeżeli nie — wykonana zostanie część po delay.

14. Ograniczony czas oczekiwania po stronie wywołującej

Przydatne np. dla samochodu, który nie chce czekać na rozładunek w nieskończoność.

```
select

   Stanowisko.Rozladuj(Dostawa);

or

   delay 5.0;
   Put_Line("Samochód zrezygnował z oczekiwania");

end select;
```

Jeżeli spotkanie rozpocznie się przed upływem 5 sekund, dostawa zostanie przekazana.

W przeciwnym razie samochód rezygnuje.

15. select ... else

Ta konstrukcja pozwala sprawdzić, czy spotkanie może zostać wykonane **natychmiast**.

```
select

   accept Napraw(Id : Integer) do
      Put_Line("Naprawiam robota");
   end Napraw;

else

   Put_Line("Brak robotów do naprawy");

end select;
```

Część else jest wykonywana, jeżeli w danym momencie nie ma oczekującego wywołania Napraw.

Uwaga: taki fragment umieszczony w bardzo szybkiej pętli może generować ogromną liczbę komunikatów.

Lepiej np.:

```
else
   delay 0.5;
end select;
```

16. Warunki when (dozory)

Alternatywa select może być aktywna tylko wtedy, gdy spełniony jest warunek.

Przykład magazynu:

```
select

   when Stan < Pojemnosc =>
      accept Przyjmij(Ilosc : Integer) do
```

```
         Stan := Stan + Ilosc;
      end Przyjmij;

or

   accept Sprawdz_Stan(Wynik : out Integer) do
      Wynik := Stan;
   end Sprawdz_Stan;

end select;
```

Jeżeli magazyn jest pełny, alternatywa Przyjmij jest wyłączona.

Można mieć kilka warunków:

```
select

   when Stan_A > 0 =>
      accept Wydaj_A(...);

or

   when Stan_B > 0 =>
      accept Wydaj_B(...);

or

   accept Stan(...);

end select;
```

17. select ... then abort

Ta konstrukcja pozwala przerwać operację, jeżeli nastąpi określone zdarzenie.

Przykład:

```
select
   delay 5.0;
   Put_Line("Operacja trwa zbyt długo");

then abort

   Bardzo_Dluga_Operacja;

end select;
```

Jeżeli Bardzo_Dluga_Operacja zakończy się przed upływem 5 sekund — wszystko jest w porządku. Jeżeli nie, po 5 sekundach zostanie przerwana.

W projekcie można użyć tego np. przy:

- naprawie robota,
- bardzo długim oczekiwaniu,
- operacji z określonym maksymalnym czasem.

Nie należy używać tej konstrukcji tam, gdzie zwykłe select ... or delay rozwiązuje problem prościej.

18. Wiele operacji w accept

Spotkanie nie musi tylko odebrać parametrów i coś z nimi zrobić. Może być dłuższym blokiem programu:

```
accept Przyjmij(Towar : Character; Ilosc : Integer) do

   Put_Line("Rozpoczynam przyjmowanie");

   Stan := Stan + Ilosc;

   Put_Line("Towar przyjęty");

end Przyjmij;
```

Należy jednak pamiętać:

podczas wykonywania accept ... do ... end **oba zadania uczestniczą w spotkaniu!**

Dlatego nie warto wykonywać tam bardzo długich operacji.

Zamiast:

```
accept Transportuj(...) do
   delay 10.0;
end Transportuj;
```

często lepiej:

```
accept Transportuj(...) do
   -- tylko odebranie danych
end Transportuj;

delay 10.0;
```

Dzięki temu proces zlecający transport nie musi czekać przez cały czas przejazdu robota.

19. Stan przechowywany wewnątrz zadania

Zadanie może posiadać własne zmienne.

```
task body Magazyn is

   Stan_A : Integer := 0;
   Stan_B : Integer := 0;

begin

   loop

      select

         accept Przyjmij_A(Ilosc : Integer) do
            Stan_A := Stan_A + Ilosc;
         end Przyjmij_A;

      or

         accept Przyjmij_B(Ilosc : Integer) do
```

```
            Stan_B := Stan_B + Ilosc;
         end Przyjmij_B;

      end select;

   end loop;

end Magazyn;
```

Takie zmienne są dostępne tylko dla magazynu. Inne zadania muszą odczytać lub zmienić stan poprzez spotkanie - jest to bardzo wygodny i bezpieczny sposób synchronizacji.

20. Tablice

Stan kilku produktów można przechowywać w tablicy.

```
type Towar_Type is range 1 .. 3;

type Stan_Magazynu is
   array (Towar_Type) of Integer;

Stan : Stan_Magazynu := (others => 0);
```

Dostęp:

```
Stan(1) := Stan(1) + 5;
```

Można także wykorzystać typ wyliczeniowy:

```
type Towar_Type is (A, B, C);

type Stan_Magazynu is
   array (Towar_Type) of Integer;
```

Wtedy:

```
Stan(A) := Stan(A) + 5;
```

jest znacznie czytelniejsze.

21. Typ wyliczeniowy

Typ wyliczeniowy bardzo dobrze nadaje się np. do stanów robota.

```
type Stan_Robota is (
   Wolny,
   Transport,
   Awaria,
   Ladowanie
);
```

Przykład:

```
Stan := Wolny;

if Stan = Awaria then
   Put_Line("Robot jest uszkodzony");
end if;
```

22. Losowanie

Do prostego losowania liczb całkowitych można użyć:

```
with Ada.Numerics.Discrete_Random;
```

Najpierw określamy zakres:

```
subtype Zakres is Integer range 1 .. 5;

package Losowanie is
   new Ada.Numerics.Discrete_Random(Zakres);
```

Generator:

```
Gen : Losowanie.Generator;
```

Inicjalizacja:

```
Losowanie.Reset(Gen);
```

Losowanie:

```
X := Losowanie.Random(Gen);
```

Cały przykład:

```
with Ada.Numerics.Discrete_Random;

procedure Test is

   subtype Zakres is Integer range 1 .. 5;

   package Losowanie is
      new Ada.Numerics.Discrete_Random(Zakres);

   Gen : Losowanie.Generator;
   X   : Zakres;

begin

   Losowanie.Reset(Gen);

   X := Losowanie.Random(Gen);

end Test;
```

23. Losowy czas

Można np. wylosować liczbę od 1 do 4:

```
X := Losowanie.Random(Gen);

delay Duration(X);
```

Wtedy zadanie zatrzyma się na losowo wybrany czas od 1 do 4 sekund.

Uwaga na generatory losowe w wielu zadaniach

Jeżeli kilka zadań potrzebuje losowania, najprościej dać **każdemu własny generator**.

Nie należy bez potrzeby współdzielić jednej zmiennej generatora pomiędzy wiele zadań.

24. Kontrolowane zakończenie zadania

Dobrze jest przewidzieć specjalne spotkanie:

```
entry Zakoncz;
```

A następnie:

```
loop

   select

      accept Transportuj(...);

   or

      accept Zakoncz;
      exit;

   end select;

end loop;

Put_Line("Robot zakończony");
```

To bardziej eleganckie niż pozostawienie zadań w nieskończonych pętlach.

25. terminate

Ada posiada również alternatywę:

```
select

   accept Transportuj(...);

or

   terminate;

end select;
```

Pozwala ona zakończyć zadanie, gdy system zadań nie potrzebuje już jego działania.

Na początku nauki jednak łatwiejsze do zrozumienia może być jednak jawne:

```
entry Zakoncz;
```

26. Zarysy zadań

Ten punkt zawiera szkielety głównych modułów programu. Są to jedynie sugestie – nie trzeba się ich dokładnie trzymać. Mogą pomóc w pierwszych krokach realizacji projektu.

a. Przykładowy szkielet robota

```
task type Robot_Type(Id : Integer) is

   entry Transportuj(
      Towar : Character;
      Ilosc : Integer
   );

   entry Zakoncz;

end Robot_Type;


task body Robot_Type is

   Koniec : Boolean := False;

begin

   while not Koniec loop

      select

         accept Transportuj(
            Towar : Character;
            Ilosc : Integer
         ) do

            Put_Line(
               "[R" & Integer'Image(Id) &
               "] otrzymałem transport"
            );

         end Transportuj;

         Put_Line(
            "[R" & Integer'Image(Id) &
            "] jadę"
         );

         delay 2.0;

         Magazyn.Przyjmij(Towar, Ilosc);

      or

         accept Zakoncz;
         Koniec := True;

      end select;

   end loop;

end Robot_Type;
```

b. Przykładowy szkielet magazynu

```
task Magazyn is

   entry Przyjmij(
      Towar : Character;
      Ilosc : Integer
   );

   entry Sprawdz(
      Wynik : out Integer
   );

   entry Zakoncz;

end Magazyn;


task body Magazyn is

   Stan   : Integer := 0;
   Koniec : Boolean := False;

begin

   while not Koniec loop

      select

         when Stan < 50 =>

            accept Przyjmij(
               Towar : Character;
               Ilosc : Integer
            ) do

               Stan := Stan + Ilosc;

               Put_Line(
                  "[MAGAZYN] Stan:" &
                  Integer'Image(Stan)
               );

            end Przyjmij;

      or

         accept Sprawdz(
            Wynik : out Integer
         ) do

            Wynik := Stan;

         end Sprawdz;

      or

         accept Zakoncz;
         Koniec := True;
```

```
      end select;

   end loop;

end Magazyn;
```

c. Przykładowy szkielet stanowiska oczekującego na zgłoszenia

```
loop

   select

      accept Nowa_Dostawa(D : Dostawa) do
         Put_Line("Przyjęto samochód");
      end Nowa_Dostawa;

   or

      delay 3.0;
      Put_Line("Brak nowych dostaw");

   end select;

end loop;
```

d. Przykładowa kolejność działania programu głównego

Program główny może:

1. utworzyć wszystkie zadania,
2. wygenerować określoną liczbę dostaw,
3. odczekać na ich przetworzenie,
4. wysłać zadaniom komunikaty Zakoncz,
5. wypisać podsumowanie.

Schemat:

```
procedure Main is

   Robot1 : Robot_Type(1);
   Robot2 : Robot_Type(2);

begin

   -- generowanie pracy

   delay 20.0;

   Robot1.Zakoncz;
   Robot2.Zakoncz;
   Magazyn.Zakoncz;

end Main;
```

Program główny nie zakończy się, dopóki zależne od niego zadania nadal działają.

27. Ryzyko zakleszczenia

a. spotkanie może blokować program lub inne procesy

Instrukcja:

```
Magazyn.Przyjmij('A', 10);
```

nie oznacza:

wyślij wiadomość i idź dalej.

Oznacza:

poczekaj, aż magazyn będzie gotowy wykonać odpowiadające accept.

Pytanie, czy będzie gotowy? Czy nie jest pełny? Dlatego niewłaściwa kolejność spotkań lub brak kontroli odpowiednich zmiennych może prowadzić do zakleszczenia.

b. Prosty przykład zakleszczenia

Zadanie A:

```
B.Wejscie_B;
accept Wejscie_A;
```

Zadanie B:

```
A.Wejscie_A;
accept Wejscie_B;
```

Oba zadania najpierw próbują wywołać spotkanie u drugiego zadania.

Oba czekają.

Żadne nie dochodzi do accept.

Program zostaje zablokowany.

c. Jak ograniczać ryzyko zakleszczeń

W projekcie warto stosować kilka prostych zasad:

- spotkania powinny być możliwie krótkie,
- nie wykonuj długiego delay wewnątrz accept,
- unikaj sytuacji, w której dwa zadania jednocześnie czekają na siebie,
- dokładnie przemyśl kolejność wywołań,
- zapewnij możliwość zakończenia każdego zadania,
- wypisuj komunikaty diagnostyczne.

28. Procesy działają współbieżnie

Kilka zadań może jednocześnie (współbieżnie) wykonywać, np. wypisywanie:

```
Put_Line(...);
```

Dlatego kolejność komunikatów nie zawsze będzie taka sama. To jest normalne, a **wręcz oczekiwane.**

Przykład:

```
[R1] jadę
[R2] rozpocząłem transport
[R1] dostarczyłem towar
```

Przy następnym uruchomieniu kolejność może być inna.

```
[R1] jadę
[R1] dostarczyłem towar
[R2] rozpocząłem transport
```

Nie zakładaj ani nie narzucaj konkretnej kolejności procesów

Błędne jest założenie:

Robot 1 na pewno wykona się przed Robotem 2.

Jeżeli kolejność ma znaczenie w projekcie, **musi wynikać z synchronizacji**.

29. Najważniejsze konstrukcje – podsumowanie

Utworzenie zadania

```
task Robot;
```

Typ zadania

```
task type Robot_Type(Id : Integer);
```

Wejście

```
entry Transportuj(...);
```

Obsługa spotkania

```
accept Transportuj(...) do
   ...
end Transportuj;
```

Kilka tak samo możliwych spotkań

```
select
   accept A;
or
   accept B;
end select;
```

Warunek

```
when Warunek =>
   accept A;
```

Timeout

```
select
   accept A;
or
```

```
   delay 2.0;
end select;
```

Operacja bez czekania

```
select
   accept A;
else
   ...
end select;
```

Przerwanie zbyt długiej operacji

```
select
   delay 5.0;
then abort
   Operacja;
end select;
```

30. Minimalny model projektu

Podczas projektowania warto myśleć o każdym zadaniu jako o osobnym pracowniku.

Na przykład:

```
SAMOCHÓD
    |
    | spotkanie
    v
STANOWISKO ROZŁADUNKOWE
    |
    | spotkanie
    v
ROBOT
    |
    | spotkanie
    v
MAGAZYN
```

Każdy z nich:

- działa niezależnie,
- posiada własny stan,
- może czekać,
- komunikuje się z innymi poprzez spotkania.

select oznacza w uproszczeniu:

„Jestem gotowy obsłużyć kilka różnych zdarzeń. Zobaczmy, które nastąpi.”

31. Co powinno pojawić się w projekcie

Pytania kontrolne przed oddaniem projektu:

- kilka zadań działa współbieżnie,
- zastosowano entry,
- zastosowano accept,

- zastosowano select ... or,
- wykorzystano co najmniej 3 wymagane odmiany select,
- when zależy od rzeczywistego stanu programu,
- występują opóźnienia symulujące czas działania
- występuje losowość
- program nie kończy się przypadkowo,
- zadania można poprawnie zakończyć,
- komunikaty terminalowe pozwalają obserwować przebieg symulacji,
- nie występuje oczywiste zakleszczenie,
- każde zadanie, warunek czy synchronizacja mają sens w kontekście wybranego scenariusza